

Кафедра «Информационные системы»

ОТЧЕТ

По СРОN№1

Тема: Учет студентов и дисциплин

По дисциплине : Проектирование и применение NoSQL-технологий

Вариант : №2

Выполнил(а):

Симиониди Анастас

Группа: ИС 24-22

Проверил(а):

Рашидинов Д.Р

[Алматы], 2026

Цель и постановка задачи

Цель работы: Разработка рабочего прототипа NoSQL-подсистемы для учета студентов, дисциплин и их успеваемости на базе документоориентированной СУБД MongoDB.

Постановка задачи:
В традиционных реляционных базах данных (например, PostgreSQL) для формирования транскрипта студента (списка всех его оценок с названиями дисциплин и кредитами) требуется выполнение множества операций соединения (JOIN) между таблицами студентов, курсов, преподавателей и регистраций. Это создает высокую нагрузку на БД при частых чтениях. Необходимо спроектировать модель данных, которая оптимизирует чтение (паттерны доступа), использует гибкую схему для карточек студентов и обеспечивает быстрый расчет агрегатов (GPA, средние баллы), а также реализовать работающий прототип с тестами.

2. Требования

2.1 Функциональные требования (FR)

ID, Требование

FR-1, Вести карточку студента: ФИО, номер зачётки, программа, курс, статус, контакты.

FR-2, Вести справочник дисциплин: код, название, кредиты, кафедра, семестр, преподаватели.

FR-3, Вести справочник преподавателей и связывать их с дисциплинами.

FR-4, Регистрировать студента на дисциплину в семестре.

FR-5, Выставлять оценку (балл 0–100, буква, grade points).

FR-6, Выдавать транскрипт студента со всеми оценками.

FR-7, Считать агрегаты: средний балл по дисциплине, GPA студента (взвешенный).

FR-8, Находить студентов с задолженностью (оценка < 60).

2.2 Нефункциональные требования (NFR)

ID, Требование

NFR-1, Производительность: выборка профиля и транскрипта ≤ 50 мс на 10^4 документах (за счет индексов).

NFR-2, Целостность: уникальность зачеток, кодов дисциплин и пар (студент, дисциплина, семестр).

NFR-3, Историчность: оценки не должны «плыть» при переименовании дисциплины (snapshot).

NFR-4, Воспроизводимость: детерминированный сид, развёртывание одной командой, .env файл.

NFR-5, Безопасность: отсутствие реальных ПДн и секретов в репозитории.

3. Предметная область и Access Patterns

Были выделены ключевые паттерны доступа к данным (Access Patterns), определяющие структуру коллекций и индексов.

#, Access Pattern, Приоритет, Ожидаемый результат

AP-1, Профиль студента по номеру зачётки, High, 1 документ со встроенными контактами

AP-2, Транскрипт студента за все семестры, High, N записей, отсортированы по семестру

AP-3, Список группы по дисциплине, High, N записей с ФИО и оценкой

AP-4, Каталог дисциплин кафедры (фильтр по кредитам), Medium, Список дисциплин

AP-5, Поиск студента по префиксу ФИО, Medium, До 10 совпадений

AP-6, Статистика по дисциплине (средний балл, pass rate), Medium, 1 документ-агрегат

AP-7, GPA по всем студентам (взвешенный по кредитам), Medium, N документов с сортировкой

AP-8, Топ-N дисциплин по числу регистраций, Low, N документов

AP-9, Студенты с задолженностью (grade < 60), High, Список «студент — дисциплина»

AP-10, Нагрузка преподавателей, Low, N документов

4. Сравнение альтернатив и ADR

4.1 Сравнение СУБД

Критерий, PostgreSQL, MongoDB (выбрана), Neo4j, Cassandra

Модель, Реляционная (4NF), Документная, Графовая, Колоночная

Транскрипт (AP-2), 4 JOIN, 1 запрос по индексу, Обход, 1 партиция

Агрегации, SQL-агрегаты, Pipeline (достаточно), Слабо, Очень слабо

Масштабирование, Ограниченное, Sharding из коробки, Кластер, Отличное

Порог входа, Средний, Низкий, Средний, Высокий

4.2 Architecture Decision Record (ADR-001)

Контекст: Требуется быстрый доступ к транскриптам, гибкая схема для полей студента и мощные агрегации.
Решение: Выбрана MongoDB.
Обоснование:

Транскрипт (AP-2) в MongoDB читается одним запросом (без JOIN), так как данные о дисциплине встроены прямо в регистрацию.

Aggregation pipeline полностью покрывает AP-6–AP-10 ($facet, $bucket, $group).

Гибкая схема позволяет добавлять новые поля в карточку студента без миграций.
Последствия: Денормализация данных (создание snapshot-копий) для обеспечения историчности (NFR-3).

4.3 ADR-002: Embedding vs Referencing

Контакты студента, расписание — Embedding (встроены), так как они всегда читаются вместе и имеют ограниченный размер.

Преподаватели — Referencing (связь), так как они существуют независимо.

Регистрации (Оценки) — Referencing + Snapshot. Вынесены в отдельную коллекцию (так как их много), но снимок названия дисциплины и ФИО студента встроен для историчности.

5. Схема данных и архитектура

Архитектура проекта представляет собой паттерн «Слой доступа к данным» (Data Access Layer). Python-скрипты (src/demo.py, tests/) обращаются к модулю src/queries.py, который использует pymongo для отправки запросов в MongoDB.

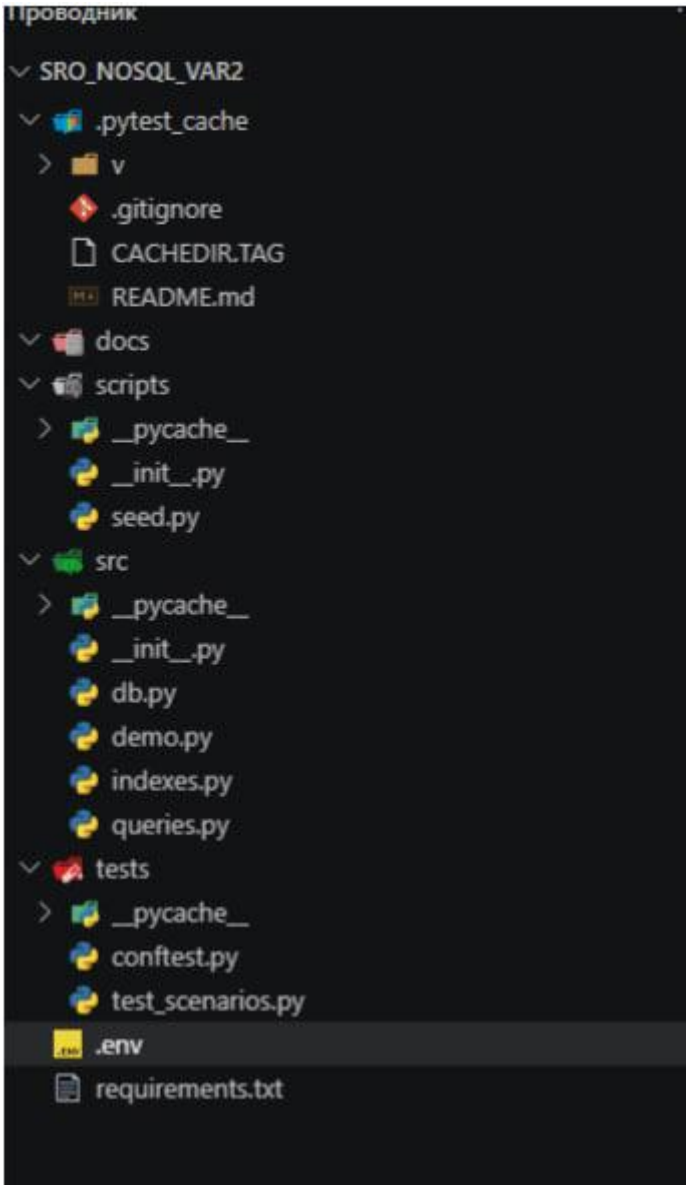


Рис 1.1 Стуркута проекта

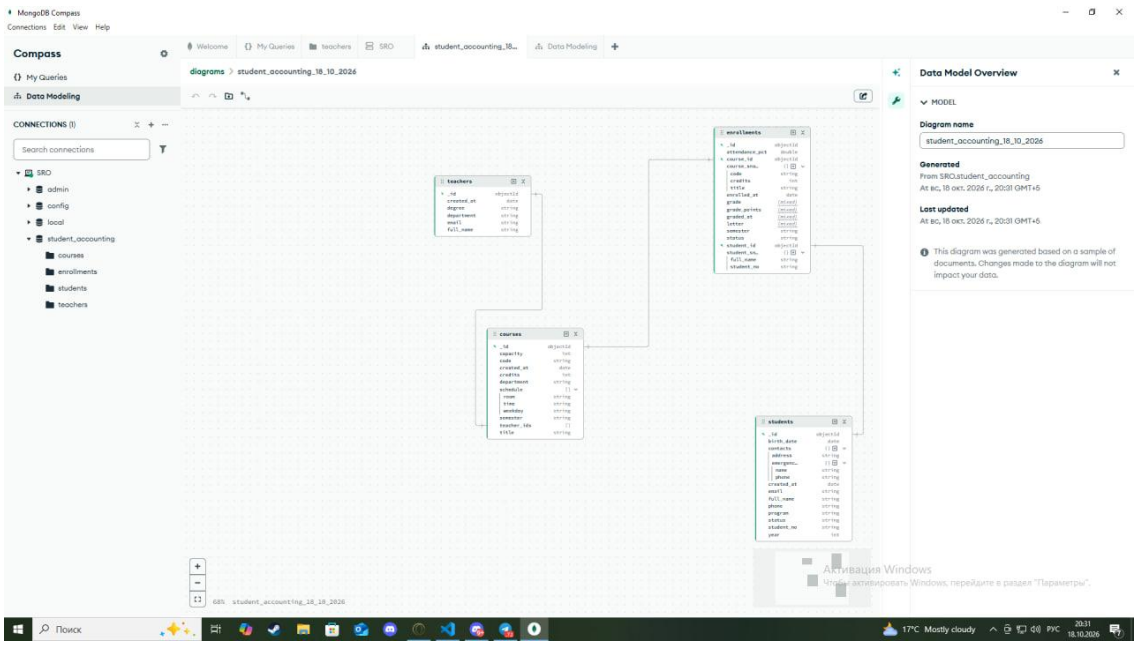


Рис 1.2 Диаграмма базы данных NoSql

Пример документа коллекции enrollments (связь + snapshot):

json

```
{
  "_id": ObjectId("..."),
  "student_id": ObjectId("..."),
  "course_id": ObjectId("..."),
  "semester": "2024-Spring",
  "status": "completed",
  "grade": 87,
  "letter": "B+",
  "grade_points": 3.5,
  "student_snapshot": { "student_no": "IS-2022-001", "full_name": "Алексеев Артём"
},
  "course_snapshot": { "code": "IS-201", "title": "NoSQL-технологии", "credits": 4 }
}
```

Индексы: Для обеспечения высокой скорости чтения созданы индексы, в том числе уникальные (student_no, code), составные (student_id + semester) и уникальные составные (student_id + course_id + semester), защищающие от дублей.

6. Реализация и фрагменты кода

Проект реализован на Python 3 с использованием библиотеки PyMongo.

Генерация тестовых данных (seed.py): Скрипт детерминированно создает 6 преподавателей, 8 дисциплин, 20 студентов и ~80 регистраций, формируя snapshot для каждой регистрации.

Фрагмент кода: Расчет GPA (взвешенный по кредитам)
Реализован через Aggregation Pipeline. Сначала отбираются завершенные курсы, затем группируются по студенту, умножая grade points на кредиты, и делится на общую сумму кредитов.

python

```
pipeline = [
    {"$match": {"grade": {"$ne": None}}},
    {"$group": {
        "_id": "$student_id",
        "student_no": {"$first": "$student_snapshot.student_no"},
        "total_credits": {"$sum": "$course_snapshot.credits"},
        "weighted_points": {"$sum": {"$multiply": ["$grade_points",
"$course_snapshot.credits"]}},
    }},
    {"$addFields": {
        "gpa": {"$round": [{"$divide": ["$weighted_points", "$total_credits"]}, 2]}
    }},
    {"$sort": {"gpa": -1}}
]
```

7. Запросы и результаты

В рамках работы реализовано 10 демонстрационных запросов (Access Patterns).

Простые запросы (поиск профиля, транскрипт) выполняются за доли миллисекунд благодаря индексам.

Нетривиальные запросы:

AP-6 (Статистика по дисциплине): Использует $facet для параллельного вычисления среднего балла, процента сдавших и распределения оценок (через $bucket).

AP-7 (GPA): Агрегация $group с математическим взвешиванием оценок на кредиты.

AP-9 (Задолженности): Использует $lookup для соединения коллекции enrollments с students и вывода ФИО должников.

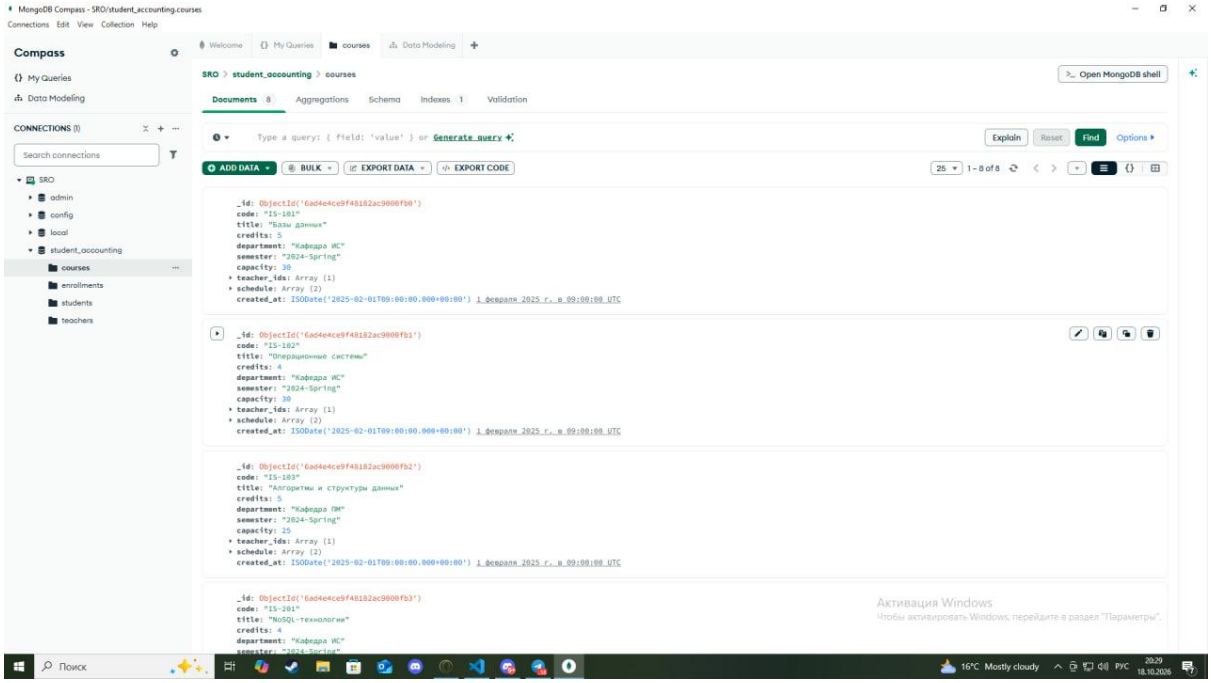


Рис 1.3 данные о дисциплинах

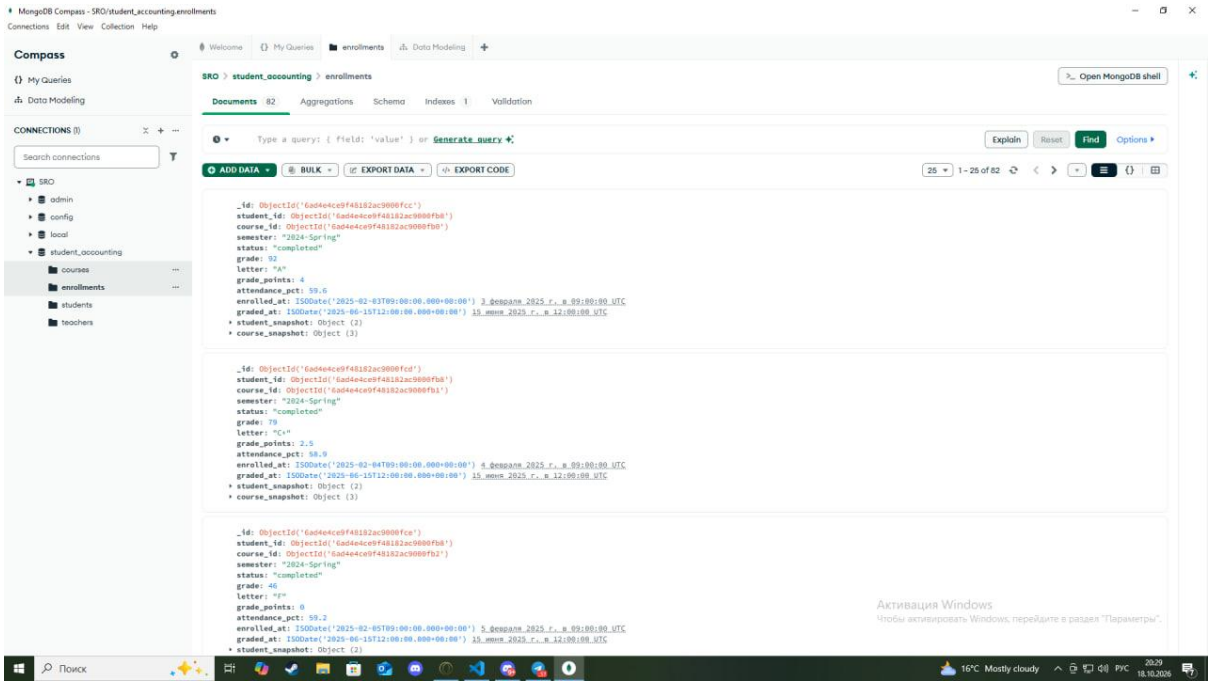


Рис 1.4 Данные о зачисление

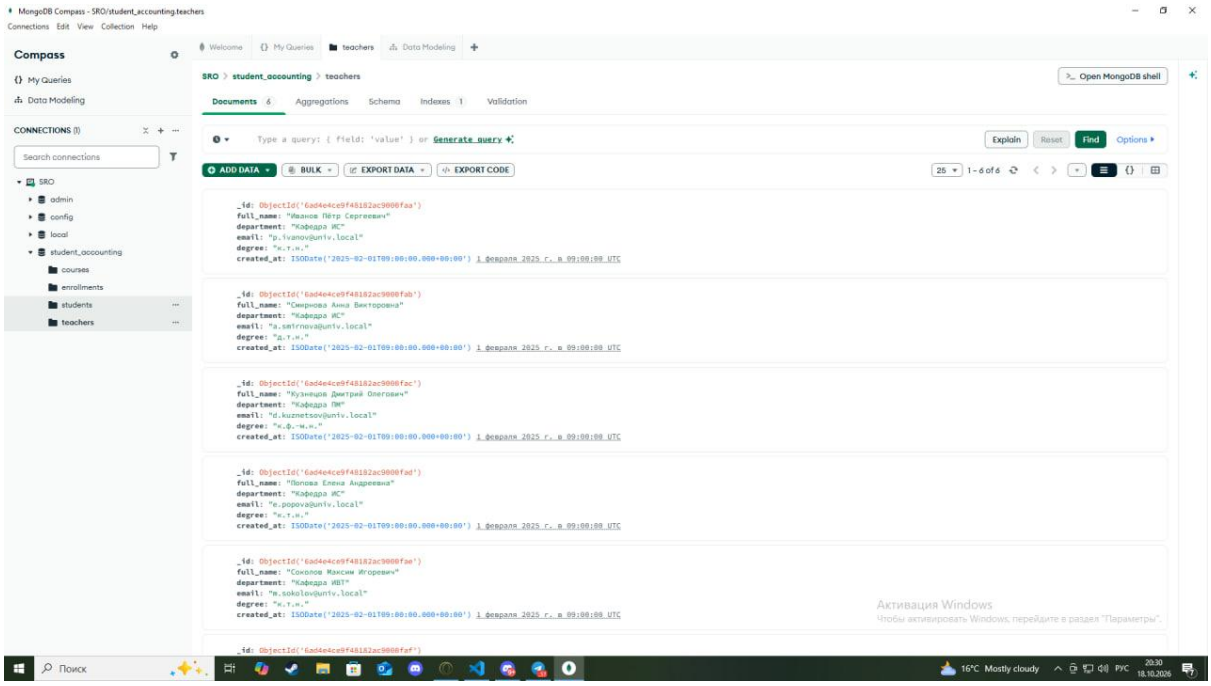


Рис 1.6 Данные о студентах

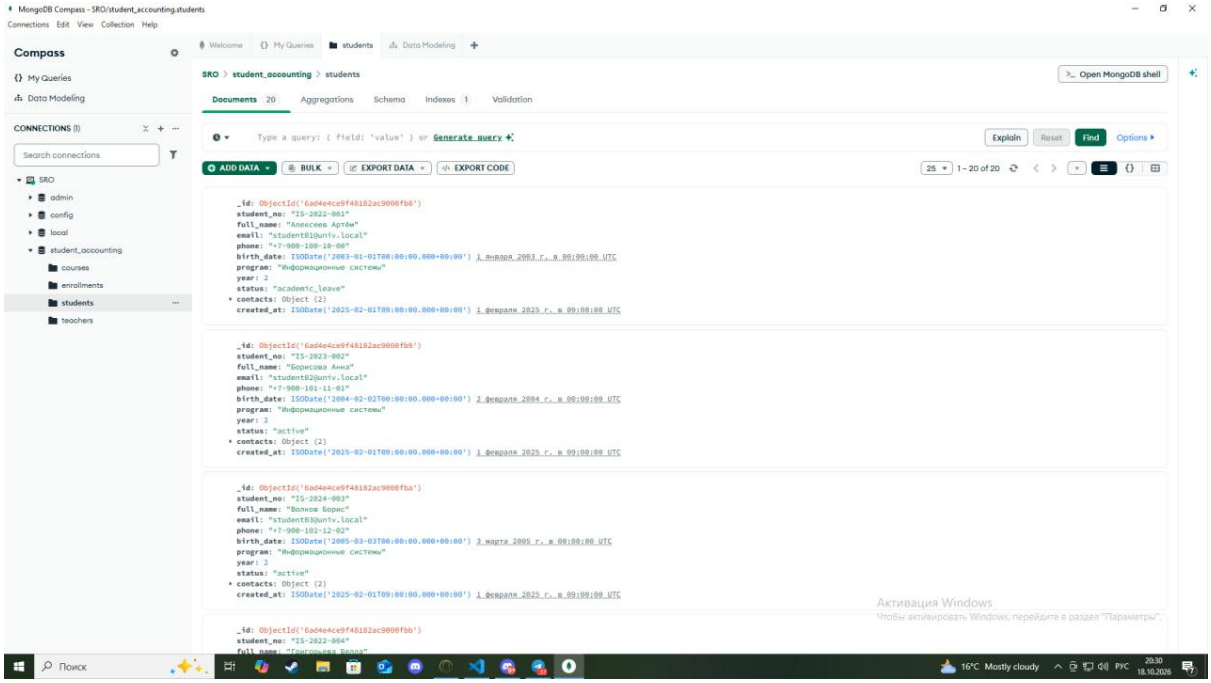


Рис 1.7 данные о учителях

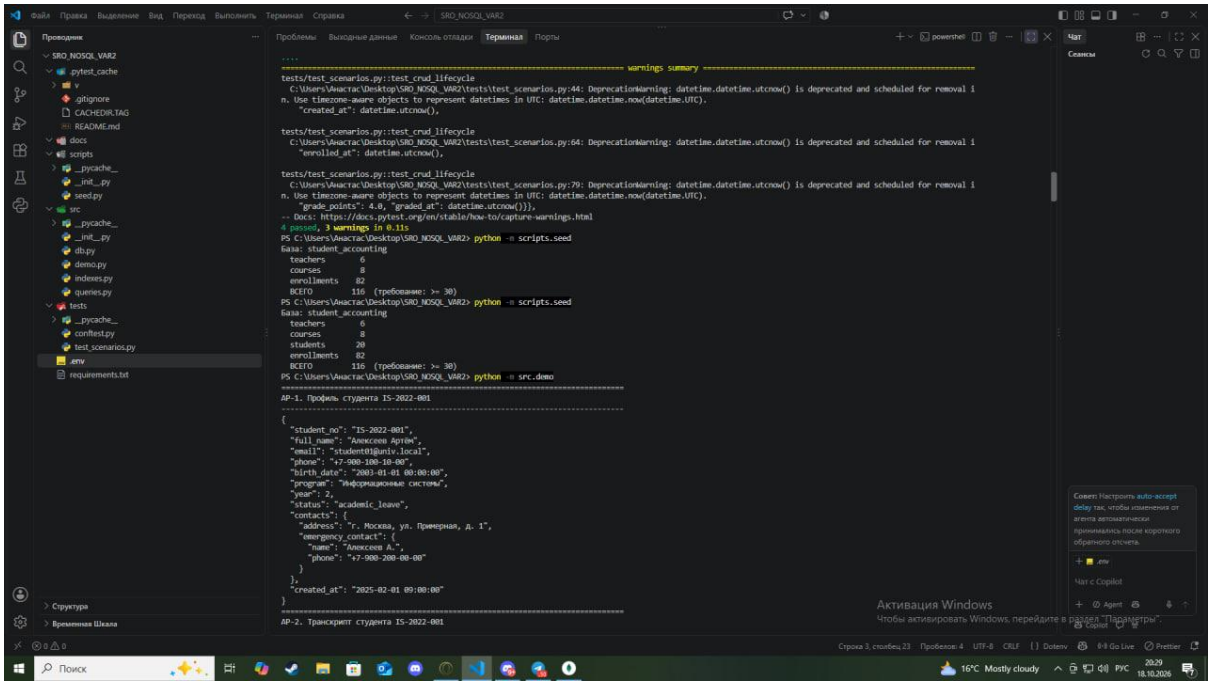


Рис 1.8 логи из терминала показывющие что все работает так как надо

Сравнение до и после индексации:
При выполнении поиска по полю student_no (для которого создан уникальный индекс) MongoDB использует стадию IXSCAN (Index Scan). Если выполнить поиск по неиндексированному полю (например, email), СУБД вынуждена использовать COLLSCAN (полный перебор коллекции), что на больших данных значительно медленнее.

8. Тестирование

Для проверки работоспособности написано 4 сценария с использованием фреймворка pytest:

test_seed_volume_and_uniqueness: Проверяет, что база наполнена (≥30 записей) и что уникальные индексы работают (выбрасывается DuplicateKeyError при попытке вставить дубликат зачетки).

test_crud_lifecycle: Проверяет полный цикл: создание студента, чтение, обновление статуса, запись на курс, выставление оценки, пересчет GPA и удаление.

test_gpa_matches_manual_calculation: Сравнивает результат агрегации GPA из базы данных с ручным расчетом, произведенным средствами Python.

test_snapshot_isolation_and_index_usage: Доказывает, что при изменении названия курса в справочнике, в старых транскриптах остается историчное название (snapshot). Также проверяет, что запросы идут через индекс (IXSCAN).

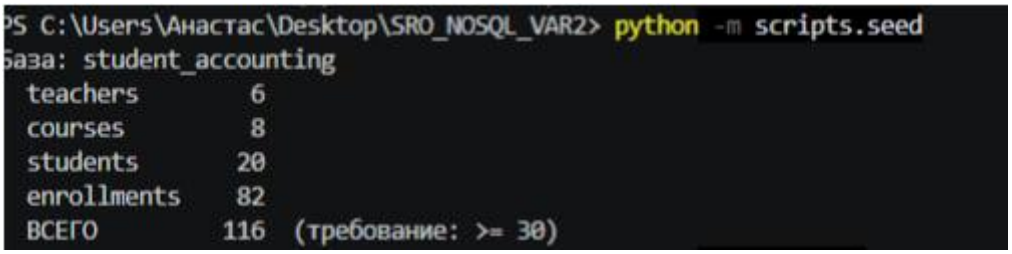


Рис 1.9 общее количество записей

9. Ограничения

В тестовом наборе данных представлен только один семестр (2024-Spring).

Прототип не имеет системы аутентификации и авторизации.

Snapshot-данные в регистрациях намеренно не синхронизируются при изменении справочника дисциплин, чтобы сохранить историческую достоверность оценок (требование NFR-3).

10. Выводы

В ходе выполнения мини-проекта была спроектирована и реализована NoSQL-подсистема на базе MongoDB. Использование документной модели позволило избавиться от сложных JOIN-запросов при формировании транскриптов, а встроенный Aggregation Framework успешно справляется с аналитикой (расчет GPA, статистика). Выбор между встраиванием и связыванием (embed/ref) был обоснован через ADR. В результате получен работающий, покрытый тестами прототип, готовый к горизонтальному масштабированию.

11. Ссылка на GitHub

Репозиторий проекта: https://github.com/Anastas-Simionidi/No_sql.git